WayCore + WayHat

Manual de uso, instalación y conexiones

Asistente Karbys · ESP32 · Bluetooth SPP · Gemini

Proyecto	WayCore — cerebro digital; WayHat — dispositivo físico de apoyo
Asistente	Karbys (se pronuncia «guaycor» / Karbys, «guayjat» para WayHat)
App Android	com.wayhat.waycore · versionCode 6 · versionName 0.3.0
Firmware	WayHat_v6_WayCore.ino · ESP32-WROOM-32 · Bluetooth Classic
Enlaces	Bluetooth SPP (teléfono <-> WayHat) e Internet HTTPS (teléfono <-> Gemini)
Fecha	5 de octubre de 2026

Este documento explica cómo conectar el hardware, cómo dejar la app lista para funcionar y cómo utilizarla en el día a día. Todo lo descrito aquí corresponde al código del repositorio **waycore30**.

> Nunca compartas tu clave de Gemini. Se guarda únicamente en el archivo local.properties, que no debe subirse al repositorio.

Contenido

El manual está pensado para leerse en orden la primera vez (instalación) y para consultarse por secciones después (uso diario y referencia).

1. Introducción 5

1.1 Qué es WayCore 5

1.2 Componentes del sistema 5

1.3 Cómo leer este manual 5

2. Arquitectura general y flujo de datos 6

2.1 Recorrido de una consulta por voz 6

2.2 Regla de oro sobre los datos 6

3. Conexiones de hardware (WayHat / ESP32) 7

3.1 Lista de materiales 7

3.2 Mapa de pines 7

3.3 Diagrama de cableado 8

3.4 Cableado detallado por módulo 8

3.5 Notas eléctricas importantes 9

4. Conexiones lógicas y de red 10

4.1 Enlace Bluetooth SPP (teléfono <-> WayHat) 10

4.2 Protocolo JSON entre WayCore y WayHat 10

4.3 Conexión con Gemini 12

4.4 Servicios internos de la app 12

4.5 Permisos de Android 12

5. Requisitos previos 14

5.1 Para el teléfono . . . 14
5.2 Para el equipo de desarrollo . . . 14
5.3 Para el hardware . . . 14
6. Instalación y puesta en marcha . . . 14
6.1 Configurar la clave de Gemini . . . 14
6.2 Cargar el firmware en el ESP32 . . . 14
6.3 Compilar e instalar la app . . . 15
6.4 Vincular WayHat por Bluetooth . . . 15
6.5 Primer arranque y permisos . . . 15
7. Uso de la aplicación . . . 16
7.1 Pantalla principal . . . 16
7.2 Hablar con Karbys . . . 16
7.3 Comandos de voz disponibles . . . 16
7.4 Recordatorios y alarmas . . . 17
7.5 Avisos de proximidad y vibración . . . 17
7.6 Servicios en segundo plano . . . 17
8. Referencia rápida . . . 18
8.1 Puesta en marcha en 6 pasos . . . 18
8.2 Resumen de pines de WayHat . . . 18
8.3 Ciclos de tiempo del firmware . . . 18
9. Solución de problemas . . . 18
10. Seguridad, límites y buenas prácticas . . . 20
10.1 Seguridad del sistema . . . 20

10.2 Límites actuales 20

10.3 Buenas prácticas 20

11. Créditos 20

Anexo A — Mensajes de estado del enlace Bluetooth 21

Anexo B — Glosario 21

1. Introducción

1.1 Qué es WayCore

WayCore es la aplicación Android que funciona como centro de control del proyecto. Dentro de ella vive **Karbys**, un asistente de voz en español que responde por voz (TTS) y que puede consultar el estado real del teléfono (batería, hora, ubicación) y del dispositivo externo **WayHat**, un ESP32 con sensores de distancia, temperatura y humedad, pensado para apoyar la movilidad de personas con discapacidad visual.

El proyecto está diseñado para que la interacción sea principalmente **por voz** y para que la seguridad **no dependa de Internet**: aunque no haya red o falle Gemini, WayHat sigue midiendo distancias y avisando con su buzzer de forma autónoma.

1.2 Componentes del sistema

Componente	Qué es	Dónde vive
WayCore	Aplicación Android (Compose / Material 3) con la interfaz, los servicios en segundo plano y el cliente de Gemini.	`app/src/main/java/com/wayhat/waycore/`
Karbys	El asistente: escucha la palabra de activación, reconoce la voz, consulta a Gemini y habla la respuesta.	`KarbysService.kt`
WayHat	Dispositivo físico basado en ESP32-WROOM-32. Mide distancias con 3 ultrasonidos y un TF-Luna, lee clima con un DHT11 y avisa con un buzzer.	`WayHat_v6_WayCore.ino`
Enlace Bluetooth	Servicio que mantiene el canal SPP con WayHat, recibe telemetría y envía comandos validados.	`WayHatService.kt`
Gemini	Modelo de lenguaje en la nube que razona la respuesta de Karbys y puede invocar herramientas para controlar WayHat.	`GeminiClient.kt`
Recordatorios	Alarmas programadas por voz que se guardan en preferencias y se disparan con AlarmManager.	`ReminderReceiver.kt`

1.3 Cómo leer este manual

- Las secciones 2, 3 y 4 describen las conexiones: arquitectura, cableado del hardware y enlaces lógicos.
- Las secciones 5 y 6 son de instalación y se siguen una sola vez.
- La sección 7 es el uso diario de la app.
- Las secciones 8 a 11 son de consulta y resolución de problemas.

> Si solo quieres poner el sistema a funcionar: lee la sección 3 (cableado), la 6 (instalación) y la 7 (uso).

2. Arquitectura general y flujo de datos

Hay dos enlaces independientes que **no dependen entre sí**: el canal Bluetooth entre el teléfono y WayHat, y la conexión a Internet entre el teléfono y Gemini. Si Internet falla, WayHat sigue funcionando; si WayHat no está, Karbys sigue respondiendo con la información del teléfono.

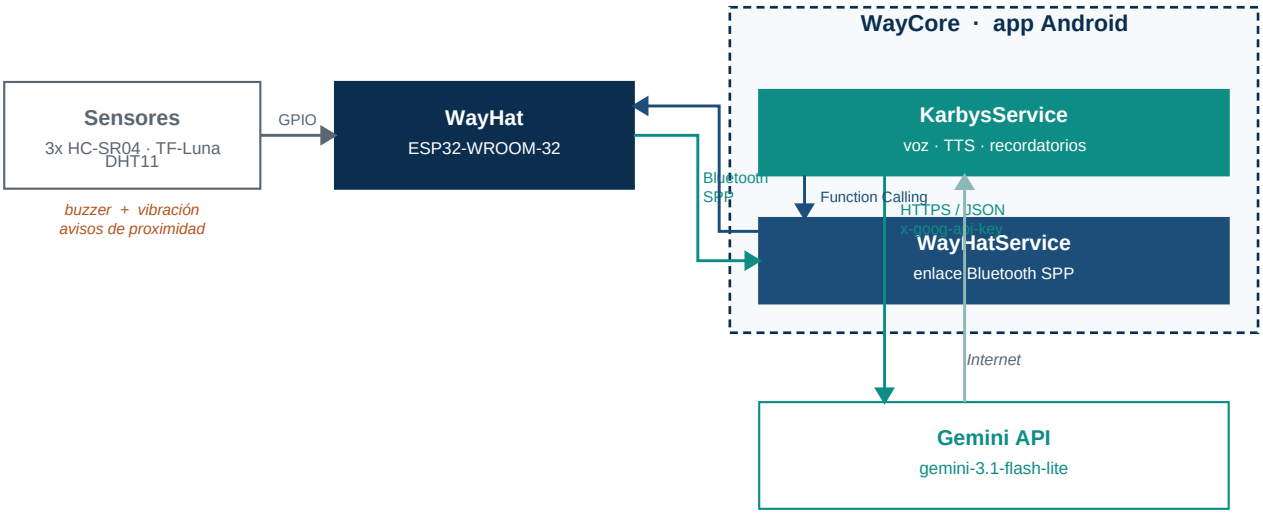


Figura 1 — Flujo de datos entre sensores, WayHat, la app WayCore y Gemini.

2.1 Recorrido de una consulta por voz

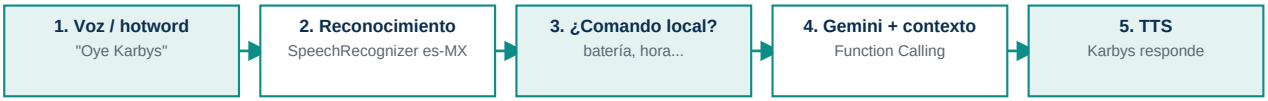


Figura 2 — Del hotword a la respuesta hablada.

- Karbys permanece en escucha pasiva del hotword con resultados parciales, en español de México (es-MX).
- Al detectar la activación emite un pitido corto (beepStart) y abre una ventana de conversación de 6 segundos.
- El texto reconocido se compara primero con una lista de comandos locales (batería, hora, fecha, ubicación, alarmas y control de WayHat). Si coincide, se responde sin salir a Internet.
- Si no coincide, se construye un contexto fresco del dispositivo y se envía a Gemini junto con las últimas 4 intervenciones de la conversación.
- Gemini puede devolver texto o una llamada a herramienta (Function Calling). La herramienta se ejecuta en WayHatService, que la valida y espera la confirmación (ack) del ESP32.
- La respuesta final se pronuncia con TTS y se reabre la ventana de conversación para seguir hablando.

2.2 Regla de oro sobre los datos

> Los valores de los sensores son la fuente de verdad. Karbys tiene instrucción explícita de no inventar ni redondear lecturas: si un sensor reporta -1, null o unavailable, debe decirlo tal cual.

3. Conexiones de hardware (WayHat / ESP32)

WayHat se construye sobre una placa **ESP32-WROOM-32** (módulo de 30 o 38 pines). El firmware usa Bluetooth Classic (SPP), por lo que **no sirve un ESP32-S2/S3/C3**: debe ser un ESP32 con Bluetooth clásico. Las conexiones siguientes se derivan directamente de las definiciones del archivo WayHat_v6_WayCore.ino.

3.1 Lista de materiales

Cant.	Elemento	Función
1	ESP32-WROOM-32 (DevKit)	Cerebro de WayHat; lee sensores, decide el buzzer y expone el puerto serie Bluetooth.
3	HC-SR04 (ultrasonido)	Distancia derecha, izquierda y trasera.
1	TF-Luna (LiDAR)	Distancia frontal por láser; se usa con una zona de seguridad más amplia.
1	DHT11	Temperatura y humedad ambiental.
1	Buzzer (activo o pasivo)	Aviso sonoro de proximidad; se controla con tone()/noTone().
1	Fuente de 5 V y cable USB	Alimentación del ESP32 y de los sensores.
—	Resistencias / divisor de nivel	Recomendadas para adaptar ECHO de 5 V a los 3.3 V del ESP32.

3.2 Mapa de pines

Esta es la referencia exacta que usa el firmware. No la cambies sin cambiar también el archivo .ino.

GPIO	Dirección	Se conecta a	Observación
25	Salida	HC-SR04 DERECHO — TRIG	Pulso de disparo de 10 us.
13	Entrada	HC-SR04 DERECHO — ECHO	Entrada 3.3 V: usar divisor de nivel.
26	Salida	HC-SR04 IZQUIERDO — TRIG	Pulso de disparo de 10 us.
14	Entrada	HC-SR04 IZQUIERDO — ECHO	Entrada 3.3 V: usar divisor de nivel.
27	Salida	HC-SR04 TRASERO — TRIG	Pulso de disparo de 10 us.
32	Entrada	HC-SR04 TRASERO — ECHO	Entrada 3.3 V: usar divisor de nivel.
16	Entrada (RX2)	TF-Luna — TX	Serial2 a 115200 8N1.
17	Salida (TX2)	TF-Luna — RX	Serial2 a 115200 8N1.
33	Bidireccional	DHT11 — DATA	Requiere resistencia pull-up de 4.7 kohm a 10 kohm.
4	Salida	Buzzer — terminal positivo	tone() a 2100 Hz; negativo a GND.
5 V / VIN	Alimentación	HC-SR04 (los 3) y TF-Luna	Los ultrasonidos necesitan 5 V.
3.3 V o 5 V	Alimentación	DHT11	Según la presentación del módulo.
GND	Tierra	Todos los módulos	Tierra común obligatoria.

3.3 Diagrama de cableado

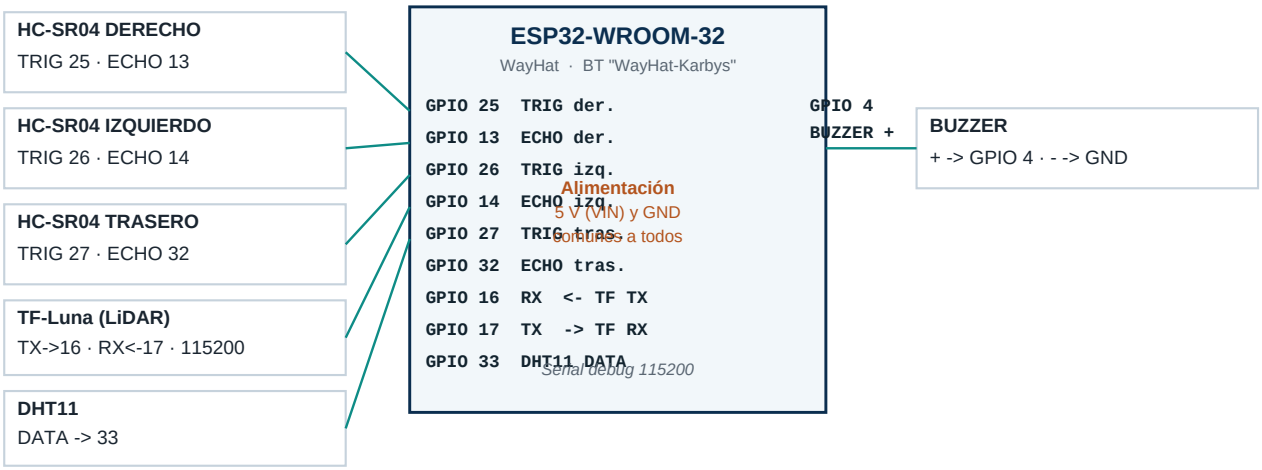


Figura 3 — Conexionado completo de WayHat. Las líneas teal representan las señales; la alimentación (5 V y GND) es común a todos los módulos.

3.4 Cableado detallado por módulo

3.4.1 Ultrasónicos HC-SR04 (derecho, izquierdo y trasero)

Cada sensor usa dos pines: el TRIG lo dispara el ESP32 y el ECHO devuelve un pulso cuya duración es proporcional a la distancia. El firmware mide hasta 25 ms de espera; si no hay eco devuelve -1.

```
HC-SR04          ESP32
--------------------------------
VCC      ->      5 V (VIN)
GND      ->      GND
TRIG     ->      GPIO 25 / 26 / 27   (der. / izq. / tras.)
ECHO     ->      divisor 5V->3.3V -> GPIO 13 / 14 / 32
```

Divisor recomendado para ECHO: resistencia de 1 kohm en serie con la señal y 2 kohm a GND (o 2.2 kohm / 3.3 kohm). Sin divisor, el pin del ESP32 recibe 5 V y puede dañarse con el tiempo.

3.4.2 TF-Luna (LiDAR frontal)

Se comunica por UART a 115200 baudios usando el puerto serie 2 del ESP32. Ojo con el cruce: el TX del sensor va al RX del ESP32 y viceversa.

```
TF-Luna          ESP32
------------------------------
VCC (5 V)  ->    5 V (VIN)
GND        ->    GND
TX         ->    GPIO 16  (RX2 del ESP32)
RX         ->    GPIO 17  (TX2 del ESP32)
```

El firmware lee tramas de 9 bytes que empiezan con los dos bytes 0x59 y valida su checksum. Si la trama no llega o es inválida, la distancia queda en -1 y el campo tf_ok se reporta como false.

3.4.3 DHT11 (temperatura y humedad)

```
DHT11            ESP32
------------------------------
VCC        ->    3.3 V o 5 V (según módulo)
DATA       ->    GPIO 33  (con pull-up 4.7k - 10k a VCC)
GND        ->    GND
```

Si la lectura falla, temp y hum se envían como null y dht_ok como false. Karbys lo reportará como dato no disponible en lugar de inventar un valor.

3.4.4 Buzzer de aviso

```
BUZZER          ESP32
-------------------------------
+          ->   GPIO 4
-          ->   GND
```

El zumbador se activa con tone() a 2100 Hz en modo SAFE cuando la distancia más cercana es menor o igual a la sensibilidad. El intervalo entre pitidos se acorta a medida que el obstáculo se acerca (de 650 ms a 35 ms según el mapeo del firmware). La prueba manual usa 2200 Hz durante 250 ms.

3.5 Notas eléctricas importantes

- Todos los módulos deben compartir GND con el ESP32; sin tierra común las lecturas son erráticas.
- Los pines del ESP32 trabajan a 3.3 V. El ECHO de los HC-SR04 entrega 5 V: usa divisor de nivel.
- Si el buzzer consume más de ~20 mA, intercala un transistor (por ejemplo 2N2222) con una resistencia de base de 1 kohm en lugar de conectarlo directo al GPIO.
- Mantén los cables del TF-Luna cortos y separados de los de los ultrasonidos para reducir ruido.
- Alimenta WayHat con una fuente estable: una caída de tensión provoca reinicios y lecturas en -1.

4. Conexiones lógicas y de red

Además del cableado, el sistema depende de tres canales de comunicación: Bluetooth SPP, el protocolo JSON propio y la API de Gemini. Esta sección documenta cada uno con los valores exactos que usa el código.

4.1 Enlace Bluetooth SPP (teléfono <-> WayHat)

Parámetro	Valor
Nombre del dispositivo	`WayHat-Karbys`
Tecnología	Bluetooth Classic, perfil Puerto Serie (SPP)
UUID del servicio	`00001101-0000-1000-8000-00805F9B34FB`
Vinculación	Debe estar emparejado desde Ajustes de Android antes de abrir la app.
Reconexión	Automática: reintenta cada 2500 ms si está desconectado y comprueba el estado cada 1000 ms si está conectado.
Estrategia de conexión	1) createRfcommSocketToServiceRecord -> 2) createInsecureRfcommSocketToServiceRecord -> 3) createRfcommSocket(1) por reflexión.
Formato de línea	Un objeto JSON por línea terminada en salto de línea (\n).
Confirmación de comandos	Cada comando incluye un id y espera un ack; si no llega en 1800 ms se informa al asistente.
Telemetría	El ESP32 envía un JSON de telemetría cada 500 ms mientras haya cliente.

> El teléfono debe aparecer en Ajustes -> Bluetooth con el dispositivo «WayHat-Karbys» ya vinculado. La app no hace descubrimiento: busca el nombre entre los dispositivos emparejados.

4.2 Protocolo JSON entre WayCore y WayHat

4.2.1 Telemetría: WayHat -> teléfono (cada 500 ms)

```
{"type":"telemetry","available":true,"uptime_ms":12345,
 "right":87,"left":-1,"rear":140,"tf":210,"closest":87,
 "temp":26.0,"hum":61.0,"dht_ok":true,"threshold":50,
 "mode":"SAFE","buzzer":true,"right_ok":true,"left_ok":false,
 "rear_ok":true,"tf_ok":true}
```

Significado de los campos:

Campo	Tipo	Significado
`type`	texto	Siempre "telemetry".
`available`	booleano	true cuando el ESP32 está operativo.
`uptime_ms`	entero	Milisegundos desde el arranque del ESP32.
`right / left / rear`	entero	Distancia en cm del HC-SR04 derecho, izquierdo y trasero. -1 = sin lectura.
`tf`	entero	Distancia en cm del TF-Luna. -1 = sin lectura.
`closest`	entero	Menor distancia válida entre los cuatro sensores. -1 = ninguna.
`temp / hum`	decimal o null	Temperatura (°C) y humedad (%) del DHT11.
`dht_ok`	booleano	true si el DHT11 respondió correctamente.
`threshold`	entero	Sensibilidad activa en cm (20 a 150).
`mode`	texto	"SAFE" con avisos de proximidad; "CHAT" silencia los avisos pero mantiene la telemetría.
`buzzer`	booleano	Estado de los avisos sonoros.
`right_ok / left_ok`	booleano	true si el sensor devolvió una distancia mayor a 0.
`rear_ok / tf_ok`	booleano	Mismo criterio para el trasero y el TF-Luna.

4.2.2 Configuración: teléfono -> WayHat

Se usa para cambiar la sensibilidad, el modo y los avisos sonoros. En el ESP32 el valor de threshold se limita al rango 20–150 y los modos solo admiten SAFE o CHAT.

```
{"type":"config","id":"k1730000000000_1","threshold":70,"mode":"SAFE","buzzer":true}
```

4.2.3 Comandos: teléfono -> WayHat

Solo existen estos nombres; cualquier otro devuelve error de lista blanca.

name	Efecto en WayHat
BUZZER_TEST	Emite un pitido de prueba de 2200 Hz durante 250 ms.
SENSORS	Fuerza una lectura inmediata de todos los sensores.
SAFE	Activa el modo seguro (avisos de proximidad encendidos).
CHAT	Activa el modo charla (silencia los avisos de proximidad).

```
{"type":"command","id":"k1730000000000_2","name":"BUZZER_TEST"}
```

4.2.4 Confirmación: WayHat -> teléfono

```
{"type":"ack","id":"k1730000000000_1","ok":true,"message":"OK"}
```

Si el comando no está permitido, ok llega como false y message indica el motivo (por ejemplo «Comando no permitido» o «Modo invalido»).

4.3 Conexión con Gemini

Parámetro	Valor
Modelo	`gemini-3.1-flash-lite`
Endpoint	`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.1-flash-lite:generateContent`
Autenticación	`x-goog-api-key: TU_CLAVE`
Método	POST con cuerpo JSON (contents + tools).
Tiempos de espera	Conexión 20 s, lectura 45 s, escritura 20 s.
Rondas de herramientas	Hasta 3 vueltas de Function Calling por consulta.
Memoria enviada	Últimas 4 intervenciones de la conversación.

El contexto que se envía a Gemini incluye la hora local, la batería del teléfono, la ubicación (latitud, longitud, precisión y antigüedad) y la telemetría completa de WayHat. El sistema le indica explícitamente que esos datos son la fuente de verdad y que debe responder en español latinoamericano pensado para ser escuchado en voz alta, sin Markdown ni emojis.

4.3.1 Herramientas disponibles (Function Calling)

Herramienta	Argumentos	Qué hace
`set_wayhat_sensitivity`	`centimeters (20-150)`	Cambia la distancia de activación de los avisos.
`set_wayhat_mode`	`mode: SAFE | CHAT`	Enciende o silencia los avisos de proximidad.
`set_wayhat_alerts`	`enabled: true | false`	Activa o desactiva el buzzer de WayHat.
`test_wayhat_alert`	`(ninguno)`	Pitido corto de prueba.
`refresh_wayhat_telemetry`	`(ninguno)`	Pide una lectura inmediata de sensores.

> Lista blanca estricta: Gemini no puede enviar comandos Bluetooth arbitrarios. Si pide algo fuera de estas cinco herramientas, WayHatService lo rechaza y Karbys informa de ello.

4.4 Servicios internos de la app

Servicio	Tipo	Función
`KarbysService`	Primer plano · micrófono + ubicación	Reconocimiento de voz, TTS, memoria corta, recordatorios y monitor de batería baja. Notificación persistente «Karbys está activo» (id 2401).
`WayHatService`	Primer plano · dispositivo conectado	Mantiene el socket SPP, recibe telemetría, ejecuta las herramientas permitidas y gestiona la vibración de seguridad. Notificación «Enlace Bluetooth con WayHat» (id 2601).
`WayCoreService`	Primer plano	Servicio auxiliar declarado en el manifiesto. Actualmente MainActivity no lo arranca: se conserva como base para futuras funciones.
`ReminderReceiver`	BroadcastReceiver	Recibe la alarma de un recordatorio y despierta a KarbysService para que lo anuncie por voz.

4.5 Permisos de Android

La app pide en tiempo de ejecución solo los permisos estrictamente necesarios; el resto quedan declarados en el manifiesto.

Permiso	Se pide al usuario	Para qué sirve
`RECORD_AUDIO`	Sí	Reconocimiento de voz y hotword de Karbys.
`ACCESS_COARSE_LOCATION`	Sí	Ubicación aproximada para el contexto de Karbys.

Permiso	Se pide al usuario	Para qué sirve
`ACCESS_FINE_LOCATION / BACKGROUND_LOCATION`	Declarados	Ubicación precisa y en segundo plano.
`BLUETOOTH_CONNECT / BLUETOOTH_SCAN`	Sí, en Android 12 o superior	Conectar y buscar el dispositivo WayHat.
`POST_NOTIFICATIONS`	Sí, en Android 13 o superior	Mostrar las notificaciones de los servicios en primer plano.
`INTERNET`	No (automático)	Llamadas a la API de Gemini.
`FOREGROUND_SERVICE (+ tipos)`	No (automático)	Mantener Karbys y el enlace Bluetooth activos.
`WAKE_LOCK / VIBRATE`	No (automático)	No suspender el servicio y vibrar en los avisos de proximidad.

Requisito de sistema: **Android 8.0 (API 26) o superior**; compilado con la API 35. Se recomienda un teléfono con Bluetooth clásico y motor de TTS en español instalado.

5. Requisitos previos

5.1 Para el teléfono

- Android 8.0 (API 26) o superior.
- Bluetooth clásico disponible y activado.
- Conexión a Internet para las respuestas de Gemini.
- Motor de texto a voz en español instalado (los servicios de Google TTS suelen incluirlo).
- Una clave de API de Gemini válida (Google AI Studio).

5.2 Para el equipo de desarrollo

- Android Studio (Koala o superior) con el plugin de Kotlin y el Android SDK 35.
- JDK 17 (el proyecto compila con sourceCompatibility y targetCompatibility 17).
- Arduino IDE 2.x con el soporte de placas ESP32 (Espressif Systems) instalado.
- Librería DHT sensor library de Adafruit (junto con Adafruit Unified Sensor) para compilar el firmware.
- Cable USB de datos (no solo de carga) para flashear el ESP32.

5.3 Para el hardware

- ESP32-WROOM-32 con Bluetooth clásico.
- Sensores, buzzer y cableado indicados en la sección 3.
- Fuente de 5 V estable o batería con capacidad suficiente.

6. Instalación y puesta en marcha

El orden recomendado es: preparar la clave, cargar el firmware, instalar la app y por último vincular el Bluetooth. Así, cuando emparejes WayHat, la app ya estará lista para conectarse.

6.1 Configurar la clave de Gemini

La clave **no** está incluida en el repositorio. Se inyecta en tiempo de compilación en el campo BuildConfig.GEMINI_API_KEY. El archivo local.properties **no debe subirse a Git**.

```
# 1. Copia la plantilla
cp local.properties.example local.properties

# 2. Edita local.properties y sustituye el valor
GEMINI_API_KEY=TU_CLAVE_DE_GEMINI
```

No pongas comillas ni espacios alrededor de la clave. El script de Gradle limpia automáticamente comillas y saltos de línea accidentales, pero lo ideal es escribirla tal cual.

También puedes pasarla por línea de comandos o como propiedad de Gradle con el mismo nombre GEMINI_API_KEY; Gradle la detecta con providers.gradleProperty antes de leer local.properties.

> Si compilas sin clave, la app se instala igual, pero Karbys responderá «Falta configurar la clave de Gemini en WayCore» en cada consulta que no sea un comando local.

6.2 Cargar el firmware en el ESP32

- Abre WayHat_v6_WayCore.ino con Arduino IDE.
- Instala el soporte de placas ESP32 desde el Gestor de placas y selecciona «ESP32 Dev Module».

- Instala la librería DHT sensor library (Adafruit) y Adafruit Unified Sensor si te las pide.
- Conecta el ESP32 por USB y selecciona el puerto serie correspondiente.
- Deja la velocidad del monitor serie en 115200 baudios y sube el sketch.
- Al terminar, abre el monitor serie: debe imprimir «Bluetooth SPP listo: WayHat-Karbys» y «WAYHAT -> WAYCORE LISTO».

Mensajes útiles del monitor serie: «WAYHAT: CELULAR CONECTADO» y «WAYHAT: CELULAR DESCONECTADO» confirman que el teléfono se está enlazando por Bluetooth.

6.3 Compilar e instalar la app

El repositorio no incluye el wrapper de Gradle, así que la forma más directa es abrirlo con Android Studio.

- Abre Android Studio y elige File -> Open seleccionando la carpeta del repositorio.
- Espera a que Android Studio sincronice Gradle (usa Android Gradle Plugin 8.7.3 y Kotlin 2.0.21).
- Conecta el teléfono con la depuración USB activada o usa un emulador con Bluetooth real (para las pruebas de WayHat hace falta un dispositivo físico).
- Pulsa Run. El módulo es :app y el paquete es com.wayhat.waycore.

Si prefieres línea de comandos, genera primero el wrapper y compila:

```
gradle wrapper --gradle-version 8.9
./gradlew assembleDebug          # APK en app/build/outputs/apk/debug/
./gradlew installDebug           # instala en el dispositivo conectado
```

> Comprueba que local.properties existe antes de compilar; de lo contrario BuildConfig queda con la clave vacía y Karbys no podrá responder.

6.4 Vincular WayHat por Bluetooth

- Enciende WayHat (ESP32 alimentado).
- En el teléfono entra en Ajustes -> Bluetooth y activa el Bluetooth.
- Busca «WayHat-Karbys» en dispositivos disponibles y empareja.
- Si el sistema pide un PIN, los valores habituales son 0000 o 1234.
- Abre WayCore: el servicio intentará la conexión SPP automáticamente.

La app no escanea dispositivos nuevos: solo busca el nombre exacto entre los ya emparejados. Si cambias el nombre en el firmware, debes cambiar también DEVICE_NAME en WayHatService.kt.

6.5 Primer arranque y permisos

- Al abrir WayCore por primera vez se solicitan micrófono, ubicación aproximada, Bluetooth y notificaciones (según la versión de Android).
- Concede el micrófono: sin él Karbys queda en pausa y aparece el botón ACTIVAR KARBYS.
- Concede Bluetooth en Android 12 o superior; de lo contrario el estado será «Permiso Bluetooth pendiente».
- Concede notificaciones para que los servicios en primer plano se muestren correctamente.
- Si niegas un permiso por error, puedes reactivarlo desde Ajustes del sistema o con el botón ACTIVAR KARBYS dentro de la app.

7. Uso de la aplicación

La pantalla principal está pensada para ser legible y accionable con lector de pantalla: cada control tiene descripción de contenido y la información importante se muestra como texto.

7.1 Pantalla principal

Elemento	Qué hace
Título WAYCORE y estado de WayHat	Indica si WayHat está conectado o muestra el último mensaje del enlace (por ejemplo «Vincula WayHat-Karbys en Bluetooth»).
Botón grande HABLAR	Abre una escucha directa sin esperar el hotword. Si Karbys está en pausa, el botón muestra KARBYS y sirve para activarlo.
Campo de texto + ENVIAR	Permite escribir la pregunta en lugar de dictarla. Útil en lugares con ruido.
Modo SEGURO / CHARLA	SEGURO mantiene los avisos de proximidad; CHARLA los silencia para conversar, pero mantiene la telemetría y las lecturas.
Deslizador de sensibilidad	Ajusta la distancia de aviso entre 20 y 100 cm, en pasos de 5 cm. El firmware acepta hasta 150 cm por comando de voz o desde Gemini.
Interruptor de avisos sonoros	Activa o desactiva el buzzer de WayHat. No afecta a la voz de Karbys.
Lecturas de distancia	Derecha, izquierda, atrás, TF-Luna y la distancia más cercana. Un guion «—» significa sin lectura válida (-1).
Batería y GPS	Nivel de batería del teléfono y última ubicación conocida en formato latitud, longitud.
RECONECTAR WAYHAT	Fuerza un intento de conexión inmediato sin esperar el ciclo automático de 2.5 s.
PROBAR BUZZER	Envía BUZZER_TEST: un pitido de 2200 Hz de 250 ms para verificar el zumbador.
ACTUALIZAR	Envía SENSORS: obliga al ESP32 a releer todos los sensores al instante.
ACTIVAR KARBYS	Aparece solo si Karbys está en pausa por falta de permiso de micrófono.

7.2 Hablar con Karbys

Karbys escucha de tres formas:

- Por palabra de activación: di «Oye Karbys» seguido de tu pregunta.
- Con el botón HABLAR de la pantalla principal.
- Escribiendo la pregunta en el campo de texto y pulsando ENVIAR.

El detector acepta variantes de la palabra de activación y del nombre, porque el reconocedor suele cambiar la ortografía: oye, hey, ey, ei, oiga, hola + karbys, karvis, karbis, carvis, carbis, carbys, karvys, carvys, karby, karvy, carby, carvy. También se acepta el nombre solo.

Después de cada respuesta, Karbys queda a la escucha durante 6 segundos (silencio máximo de 4.5 s) para que puedas continuar la conversación sin repetir la palabra de activación. Al terminar la ventana vuelve al modo de espera del hotword.

> Si hay un auricular Bluetooth o con cable conectado, Karbys redirige el audio al auricular para escuchar y hablar mejor.

7.3 Comandos de voz disponibles

Estos comandos se resuelven en el propio teléfono, sin depender de Gemini ni de Internet:

Di algo como...	Respuesta de Karbys
¿Cuánta batería tengo?	Indica el porcentaje de batería del teléfono.
¿Qué hora es?	Da la hora local en formato de 12 horas.

Di algo como...	Respuesta de Karbys
¿Qué día es hoy? / ¿Qué fecha es?	Indica la fecha completa en español.
¿Dónde estoy?	Lee latitud y longitud de la última ubicación conocida.
Pausa Karbys / Silencio Karbys	Deja al asistente en pausa hasta que digas «Oye Karbys».
Recuérdame estudiar a las siete de la tarde	Programa un recordatorio por voz.
Pon una alarma en 20 minutos	Programa un recordatorio relativo.
Mis tareas / Mis recordatorios	Enumera los recordatorios pendientes.
Cancela todas las alarmas	Borra todos los recordatorios programados.
Modo seguro / Modo charla	Cambia WayHat entre SAFE y CHAT.
Activa los avisos / Activa el buzzer	Enciende los avisos sonoros de WayHat.
Desactiva los avisos / Desactiva el sonido	Apaga el buzzer de WayHat.
Sensibilidad en 80	Cambia la distancia de aviso a 80 cm.
Estado de WayHat / Sensores de WayHat	Lee la telemetría más reciente.
Actualiza los sensores	Pide una lectura inmediata al ESP32.
Prueba el buzzer	Emite el pitido de prueba.

Cualquier otra frase se envía a Gemini junto con el contexto del dispositivo.

7.4 Recordatorios y alarmas

- Formatos aceptados: «a las 7 de la tarde», «a las 7:30», «a las 8 am/pm» y «en 20 minutos» o «en 2 horas».
- Si la hora ya pasó hoy, el recordatorio se programa para el día siguiente. Si dices «mañana», se suma un día.
- Los recordatorios se guardan en las preferencias del teléfono y se disparan con AlarmManager; al sonar, Karbys emite un tono de alerta y anuncia el recordatorio por voz.
- Karbys también avisa por voz cuando la batería del teléfono baja a 5 % o menos.

> El manifiesto declara RECEIVE_BOOT_COMPLETED, pero todavía no hay un receptor de arranque implementado: los recordatorios no sobreviven a un reinicio del teléfono. Tenlo presente si vas a apagar el equipo.

7.5 Avisos de proximidad y vibración

- En modo SAFE, WayHat hace sonar el buzzer cuando la distancia más cercana es menor o igual a la sensibilidad configurada, con un intervalo que se acorta al acercarse el obstáculo.
- El teléfono vibra además cuando algún sensor entra en zona de peligro, con un mínimo de 350 ms entre vibraciones para que no resulte molesto.
- El TF-Luna usa una zona de seguridad más amplia: como mínimo 100 cm, o la sensibilidad si esta es mayor. Es intencional, porque su alcance es mayor que el de los HC-SR04.
- En modo CHAT no hay pitidos ni vibración de proximidad, pero la telemetría sigue actualizándose y puedes consultarla cuando quieras.

7.6 Servicios en segundo plano

Mientras la app está abierta o en segundo plano verás dos notificaciones permanentes: «Karbys está activo» y «Enlace Bluetooth con WayHat». Son necesarias para que el micrófono y el socket Bluetooth sigan funcionando. Si cierras la app por completo, los servicios se detienen hasta que la vuelvas a abrir.

8. Referencia rápida

8.1 Puesta en marcha en 6 pasos

```
1. Cablea WayHat según la sección 3 (pines 25/13, 26/14, 27/32, 16/17, 33, 4).
2. Sube WayHat_v6_WayCore.ino al ESP32 y verifica el monitor serie a 115200.
3. cp local.properties.example local.properties  ->  pega tu GEMINI_API_KEY.
4. Abre el proyecto en Android Studio y ejecuta Run (módulo :app).
5. Empareja 'WayHat-Karbys' en Ajustes -> Bluetooth del teléfono.
6. Abre WayCore, concede permisos y di: 'Oye Karbys, estado de WayHat'.
```

8.2 Resumen de pines de WayHat

```
DERECHO   TRIG 25   ECHO 13
IZQUIERDO TRIG 26   ECHO 14
TRASERO   TRIG 27   ECHO 32
TF-Luna   TX->16    RX<-17   (Serial2 @115200)
DHT11     DATA  33
BUZZER    GPIO  4
Bluetooth WayHat-Karbys   SPP 00001101-0000-1000-8000-00805F9B34FB
```

8.3 Ciclos de tiempo del firmware

Acción	Cadencia
Lectura de sensores	Cada 250 ms
Envío de telemetría	Cada 500 ms (solo si hay cliente Bluetooth)
Reintento de conexión	Cada 2500 ms si está desconectado
Espera de confirmación (ack)	1800 ms
Ventana de conversación	6000 ms tras la respuesta de Karbys
Silencio para cerrar la escucha	4500 ms

9. Solución de problemas

Síntoma	Causa probable	Qué hacer
Estado: «Vincula WayHat-Karbys en Bluetooth»	El dispositivo no está emparejado o el nombre no coincide.	Empareja «WayHat-Karbys» en Ajustes. Revisa BT_NAME en el .ino y DEVICE_NAME en WayHatService.kt.
Estado: «Bluetooth está apagado»	El radio Bluetooth del teléfono está desactivado.	Actívalo y pulsa RECONECTAR WAYHAT.
Estado: «Permiso Bluetooth pendiente»	Falta BLUETOOTH_CONNECT en Android 12 o superior.	Concede el permiso en Ajustes -> Aplicaciones -> WayCore -> Permisos.
«No se pudo conectar; reintentando…» de forma continua	El socket SPP está ocupado por otra app o el ESP32 se reinicia.	Cierra otras apps de terminal Bluetooth, reinicia el ESP32 y vuelve a intentarlo. La app prueba tres métodos de conexión automáticamente.
Las distancias aparecen como «—»	Lectura -1: sensor sin eco, desconectado o con error de checksum.	Revisa cableado y alimentación. Usa ACTUALIZAR para forzar una lectura y mira el monitor serie a 115200.
Temperatura o humedad siempre no disponibles	El DHT11 no responde o falta la resistencia pull-up.	Añade una pull-up de 4.7 kohm a 10 kohm en DATA y verifica VCC y GND.
El TF-Luna no da lectura	Cruce de pines o velocidad incorrecta.	Conecta TX del sensor al GPIO 16 y RX al GPIO 17, a 115200 baudios.
Karbys no responde al hotword	Falta permiso de micrófono, ruido ambiente o el reconocedor no reconoce el nombre.	Concede el micrófono, usa el botón HABLAR o el campo de texto, y habla cerca del teléfono.

Síntoma	Causa probable	Qué hacer
Karbys dice «Falta configurar la clave de Gemini»	local.properties sin GEMINI_API_KEY o compilado antes de crearlo.	Crea el archivo con tu clave, haz Clean Project y vuelve a compilar e instalar.
Karbys dice «No pude conectar con Gemini»	Sin Internet o la API devolvió un error.	Verifica la conexión y que la clave tenga el modelo habilitado; vuelve a intentarlo.
Karbys no ejecuta un cambio en WayHat	WayHat desconectado, valor fuera de rango o herramienta no permitida.	Revisa el estado del enlace. La sensibilidad debe estar entre 20 y 150 cm y el modo debe ser SAFE o CHAT.
El buzzer no suena en modo SEGURO	Avisos desactivados, distancia por encima del umbral o cableado del zumbador.	Activa el interruptor de avisos y pulsa PROBAR BUZZER para comprobar el zumbador.
La app se cierra o el asistente se detiene	El sistema limitó los servicios en segundo plano.	Desactiva la optimización de batería para WayCore y mantén la app en uso reciente.
Gran consumo de batería	Escucha continua del hotword, Bluetooth y GPS activos.	Es esperable: el hotword requiere micrófono permanente. Usa «Pausa Karbys» cuando no lo necesites.

10. Seguridad, límites y buenas prácticas

10.1 Seguridad del sistema

- Lista blanca de comandos: el ESP32 solo acepta los tipos «config» y «command» y rechaza cualquier otro con un ack en false.
- Validación de rangos: la sensibilidad se limita a 20–150 cm y el modo solo admite SAFE o CHAT.
- Confirmación: cada comando lleva un id y el ESP32 responde con un ack antes de que Karbys confirme la acción en voz alta.
- Independencia de la red: la seguridad local de WayHat funciona sin Gemini y sin Internet.
- Datos reales: el sistema instruye a Karbys para no inventar lecturas de sensores ni estados.

10.2 Límites actuales

- El deslizador de la app llega a 100 cm, aunque el firmware acepta hasta 150 cm; ese valor superior solo se alcanza por voz o mediante Gemini.
- La telemetría depende de la última trama recibida: si se pierde el enlace, la app conserva la última lectura válida en pantalla.
- Los recordatorios no sobreviven al reinicio del teléfono (no hay receptor de arranque implementado).
- WayCoreService está declarado en el manifiesto pero no se utiliza desde MainActivity.
- El número de versión no es consistente entre archivos: el README menciona v0.5.0, el Gradle declara versionName 0.3.0 con versionCode 6, y el firmware se llama v6. Conviene unificarlo.
- Karbys no controla música ni otras aplicaciones.

10.3 Buenas prácticas

- No subas local.properties al repositorio; añádelo a .gitignore (el proyecto aún no lo incluye).
- Prueba siempre el sistema en un entorno controlado antes de usarlo en la calle.
- El asistente apoya la movilidad, pero no sustituye al bastón ni a las técnicas de orientación.
- Verifica periódicamente el estado de los sensores con «Estado de WayHat».
- Mantén el ESP32 alimentado de forma estable y revisa el cableado si aparecen lecturas en -1.

11. Créditos

El proyecto fue creado con amor y cariño para todos. WayCore es un producto de WayCorp y Karbys vive dentro de él.

Persona	Rol
Danny Joel Castro Juárez	Estudiante de primer año de Desarrollo de Software en el Instituto Nacional de San Miguel Tepezontes. Lidera el desarrollo de software, WayCore y la integración de Karbys.
Dennis Alexander	Desarrollador de hardware.
Daylin Odalis	Secretaria, portavoz, documentadora y responsable de verificación de procesos.
Emely Denisse	Diseñadora y documentadora.
Licda. Gloria Yessenia Mármol de Muñoz	Agradecimiento especial por su dedicación para enseñarles esta carrera técnica.

> Este manual se genera automáticamente con docs/generar_manual.py a partir del código del repositorio. Si cambias pines, nombres, mensajes o endpoints, vuelve a generarlo para que la documentación no quede desactualizada.

Anexo A — Mensajes de estado del enlace Bluetooth

Estos son los textos que puede mostrar la parte superior de la pantalla, tal como los emite WayHatService. Sirven para diagnosticar el estado del enlace de un vistazo.

Mensaje	Qué significa
Iniciando enlace Bluetooth…	El servicio acaba de arrancar y todavía no ha intentado conectar.
Permiso Bluetooth pendiente	Falta BLUETOOTH_CONNECT en Android 12 o superior.
Este celular no tiene Bluetooth Classic	El hardware del teléfono no soporta el perfil SPP.
Bluetooth está apagado	Activa el Bluetooth del teléfono.
Vincula WayHat-Karbys en Bluetooth	No hay ningún dispositivo emparejado con ese nombre.
WayHat vinculado: abriendo conexión SPP…	Encontró el dispositivo y está abriendo el socket.
WayHat conectado por Bluetooth	Enlace establecido; la telemetría ya fluye.
No se pudo conectar; reintentando…	Fallaron los tres métodos de conexión; reintentará en 2.5 s.
WayHat desconectado; reconectando…	Se perdió el enlace durante la lectura.
WayHat aún no está conectado	Se intentó enviar un comando sin enlace activo.

Anexo B — Glosario

Término	Significado
SPP	Serial Port Profile: emulación de un puerto serie sobre Bluetooth clásico.
Telemetría	Paquete JSON con el estado completo de los sensores de WayHat.
ack	Mensaje de confirmación que devuelve el ESP32 tras ejecutar un comando.
Hotword	Palabra de activación que despierta a Karbys («Oye Karbys»).
Function Calling	Mecanismo con el que Gemini pide ejecutar una herramienta concreta del dispositivo.
TF-Luna	Sensor de distancia por láser (LiDAR) de mayor alcance que el HC-SR04.
TTS	Texto a voz: el motor que pronuncia las respuestas de Karbys.
BuildConfig	Clase generada por Gradle donde se inyecta GEMINI_API_KEY.